

Pruebas de entrada: Aviónica

Faraday Rocketry UPV

Ejercicio 2: Selección e integración de sensores

Enunciado

- Propón y justifica qué sensores consideras imprescindibles en la aviónica de un cohete sonda. Elige los modelos (ejemplo: acelerómetro HLIS200), justificando por qué.
- Integra en el mismo proyecto generado en el anterior ejercicio el código necesario para inicializar y leer los datos de los sensores previamente seleccionados a través de los buses anteriormente configurados.

1. Selección de sensores

1.1. Fundamento físico y magnitudes necesarias

La selección de sensores parte de las magnitudes que necesitamos observar durante el vuelo. Si se toma como positivo el sentido ascendente, la dinámica vertical durante la fase propulsada puede expresarse de forma simplificada como

$$m(t) \frac{dv_z}{dt} = T(t) - D(t) - m(t)g, \quad (1)$$

donde $m(t)$ es la masa del cohete, v_z su velocidad vertical, $T(t)$ el empuje, $D(t)$ la resistencia aerodinámica y g la aceleración de la gravedad. El empuje y la masa durante la combustión serán necesarios para estimar la aceleración máxima prevista y seleccionar un sensor cuyo rango de medida no se sature.

Seleccionaremos una IMU que integre un acelerómetro y un giróscopo. El primero permitirá observar la aceleración específica, y el segundo medirá las velocidades angulares del cohete. Al reunir ambas medidas en un dispositivo SPI, necesitaremos una sola señal CS para la IMU y podremos mantener la asignación de pines prevista en el ejercicio anterior. Por otra parte, la presión atmosférica permitirá estimar la altura; su evolución temporal servirá para estimar la velocidad vertical y reconocer la transición entre ascenso y descenso. Finalmente, un receptor GNSS proporcionará la posición geográfica para seguir la trayectoria y facilitar la recuperación del cohete.

A partir de estas funciones se justificarán los sensores seleccionados, sus rangos de medida y sus interfaces de comunicación con el microcontrolador.

1.2. Selección del barómetro

El barómetro medirá la presión atmosférica absoluta. Tomaremos una medida de referencia antes del lanzamiento y compararemos con ella las medidas posteriores para estimar la altura relativa del cohete. La evolución de esa altura se utilizará para detectar el apogeo y el comienzo del descenso.

Nuestra misión de referencia alcanza aproximadamente 9 km sobre el punto de lanzamiento. Si este se encontrase cerca del nivel del mar, la presión descendería desde unos 101 kPa hasta unos 31 kPa según un modelo atmosférico estándar. Como todavía no conocemos la altitud exacta de la rampa ni las condiciones atmosféricas, adoptamos provisionalmente un rango de selección de **20 a 110 kPa**. El límite inferior deja margen por debajo de la presión estimada en el apogeo; el superior cubre la presión prevista en tierra.

No bastará con que el sensor entregue un valor dentro de ese rango: comprobaremos la precisión que especifica el fabricante *cerca de 31 kPa*. Algunas fichas distinguen entre el rango en el que el sensor puede medir y otro más estrecho en el que garantiza su precisión.

En el ejercicio anterior se propuso adquirir y procesar medidas a 100 Hz, por lo que buscaremos un barómetro capaz de entregar una medida nueva aproximadamente cada 10 ms con una configuración de resolución adecuada. También estudiaremos el ruido de la medida: cerca de 9 km, una variación de altura de 1 m corresponde aproximadamente a 5 Pa de presión en la atmósfera estándar. Esta equivalencia nos permite valorar el ruido en términos de metros de altura, sin confundirlo con la exactitud absoluta de la altitud.

Por último, el dispositivo deberá ser compatible con la alimentación de la placa y con el bus I²C1 configurado en el ejercicio anterior. Su temperatura de funcionamiento se verificará cuando se conozcan las condiciones térmicas previstas dentro de la bahía de aviónica.

1.2.1. Comparación de modelos

El Cuadro 1 compara el LPS22DF de STMicroelectronics y el MS5611 de TE Connectivity. Ambos disponen de interfaz I²C; se valoran su rango de presión, las prestaciones publicadas cerca del apogeo previsto y la adquisición a la frecuencia planteada para el proyecto [3, 4].

La cadencia de lectura requiere una comprobación aparte del rango de presión. En el MS5611-01BA03 deben tenerse en cuenta las conversiones necesarias para compensar la temperatura; en el LPS22DF habrá que escoger un promediado compatible con los 100 Hz planteados. Estas configuraciones se concretarán al integrar el sensor.

1.2.2. Elección del barómetro

Seleccionamos el LPS22DF para la misión de referencia. La presión calculada para la misión de referencia se encuentra dentro del rango de operación del LPS22DF y aproximadamente 5 kPa por encima de su límite inferior de 26 kPa [2, 3].

El límite de 20 kPa planteado inicialmente nos ha servido para explorar candidatos y reservar margen, pero no es un requisito derivado del apogeo objetivo de esta misión. Que el LPS22DF no alcance dicho límite no constituye, por sí solo, un motivo para descartarlo. Además, el fabricante publica una exactitud absoluta *típica* para el intervalo de presión que incluye el apogeo estimado. En cambio, aunque el MS5611-01BA03 puede entregar lecturas a 31 kPa, esta presión queda fuera de su intervalo de precisión completa. No podemos concluir que el LPS22DF vaya a ser más exacto que cualquier unidad del MS5611 en vuelo; sí podemos justificar mejor sus prestaciones con las especificaciones disponibles para la zona de presión que necesitamos [3, 4].

La interfaz I²C del LPS22DF encaja con el bus configurado en el ejercicio anterior y su salida puede ajustarse a los 100 Hz considerados inicialmente. Al integrar el sensor habrá que definir el promediado y comprobar que se obtienen medidas nuevas con la cadencia requerida. También deberán verificarse la presión mínima y la temperatura previstas para el lugar y las condiciones de lanzamiento reales; la estimación atmosférica no sustituye esa comprobación.

Criterio	LPS22DF	MS5611
Rango de presión	26–126 kPa	1–120 kPa como rango ampliado
Presión prevista cerca del apogeo	Los 31 kPa estimados están dentro del rango, pero próximos al límite inferior	Los 31 kPa están dentro del rango ampliado
Margen provisional hasta 20 kPa	No lo cubre	Lo cubre como rango ampliado
Precisión especificada	La ficha indica una exactitud absoluta típica de $\pm 0,45$ hPa entre 26 y 126 kPa, bajo las condiciones de temperatura indicadas	El rango de precisión completa es de 45 a 110 kPa; a 31 kPa opera fuera de dicho rango
Adquisición	Dispone de una frecuencia de salida de 100 Hz, con compensación de temperatura interna. Debe escogerse un promediado compatible con esa frecuencia	Requiere programar conversiones de presión y temperatura. Con OSR 1024 para ambas, sus tiempos máximos suman 4,56 ms por pareja de conversiones, a los que hay que añadir la comunicación por I ² C
Interfaz	I ² C	I ² C

Cuadro 1: Comparación inicial de los barómetros considerados.

1.3. Selección de la IMU

La unidad de medida inercial (IMU) reunirá en un mismo dispositivo un acelerómetro y un giróscopo. El primero medirá la aceleración específica en los tres ejes y permitirá reconocer cambios en las fases de vuelo; el segundo medirá las velocidades angulares. Así podremos adquirir ambas magnitudes mediante una única conexión al bus SPI1, utilizando el CS reservado para la IMU en el ejercicio anterior.

Para escoger el rango del acelerómetro necesitamos conocer la aceleración máxima prevista durante la propulsión. En modelos publicados de cohetes anteriores de Faraday, los valores de empuje y masa permiten obtener estimaciones simplificadas del orden de 15 g [5, 6]. Son referencias preliminares: no son máximos medidos en vuelo, no incluyen los posibles golpes de la recuperación y no describen necesariamente el cohete de este año. Por ello estudiaremos una IMU que alcance al menos $\pm 32g$, sin dar todavía por demostrado que ese rango sea suficiente.

Tampoco disponemos de una velocidad angular máxima prevista para el nuevo cohete. Los datos encontrados en otros vuelos no permiten deducirla con fiabilidad, ya que el giro depende, entre otros factores, de las aletas y de la configuración del vehículo. Compararemos los rangos disponibles en los giróscopos y dejaremos su comprobación pendiente de los datos de diseño o de una simulación del equipo.

Como punto de partida para la adquisición consideraremos unas 100 muestras por segundo, coherentes con el ejercicio anterior. La frecuencia de salida de la IMU deberá permitir obtener medidas nuevas a esa cadencia. También comprobaremos su compatibilidad con la alimentación de la placa y con el bus SPI1 compartido con la memoria externa.

1.3.1. Comparación de modelos

El Cuadro 2 compara dos IMU de STMicroelectronics: la LSM6DSO32, con un acelerómetro de hasta $\pm 32 g$, y la LSM6DSV320X, que incorpora un segundo acelerómetro para aceleraciones mayores. Se examinan sus rangos, frecuencias de salida e integración mediante SPI [7, 8].

Criterio	LSM6DSO32	LSM6DSV320X
Acelerómetros	Uno de tres ejes	Dos de tres ejes: uno para aceleraciones bajas y otro para aceleraciones elevadas
Rangos de aceleración configurables	± 4 , ± 8 , ± 16 y $\pm 32 g$	Canal de baja aceleración: ± 2 , ± 4 , ± 8 y $\pm 16 g$; canal de alta aceleración: ± 32 , ± 64 , ± 128 , ± 256 y $\pm 320 g$
Rangos del giróscopo configurables	Hasta $\pm 2000^\circ/s$	Hasta $\pm 4000^\circ/s$
Frecuencia de salida	Acelerómetro y giróscopo configurables a 104 Hz	Acelerómetro de baja aceleración y giróscopo configurables a 120 Hz; el canal de alta aceleración admite frecuencias superiores
Interfaz con el microcontrolador	SPI	SPI
Implicación para el diseño	Cubre el rango preliminar de $\pm 32 g$, que deberá contrastarse con la aceleración prevista	Ofrece margen para aceleraciones mayores y permite conservar un canal de menor rango para medirlas por separado

Cuadro 2: Comparación inicial de las IMU consideradas.

La principal diferencia para esta aplicación es el margen de medida. La LSM6DSO32 permite estudiar una solución con un único acelerómetro de hasta $\pm 32 g$. La LSM6DSV320X añade un canal para aceleraciones mayores y amplía el rango disponible del giróscopo. Antes de elegir entre ellas habrá que comprobar las aceleraciones y velocidades angulares previstas para la misión.

Esa capacidad adicional del LSM6DSV320X complica la integración del software. Los dos acelerómetros entregan medidas por separado y la IMU no decide por sí sola cuál debe utilizar nuestra aplicación. Tendríamos que configurar y leer ambos canales, y programar la lógica para utilizar el de menor rango durante el movimiento habitual y recurrir al de mayor rango cuando el primero se acerque a la saturación. Aun así, la IMU seguiría ocupando una sola conexión SPI y el CS previsto en el ejercicio anterior.

1.3.2. Elección de la IMU

Seleccionamos la LSM6DSV320X. Todavía no conocemos las aceleraciones ni las velocidades angulares máximas previstas para el cohete de este año, por lo que no podemos asegurar que los rangos de $\pm 32 g$ y $\pm 2000^\circ/s$ de la LSM6DSO32 sean suficientes. La LSM6DSV320X ofrece un acelerómetro configurable hasta $\pm 320 g$ y un giróscopo de hasta $\pm 4000^\circ/s$. También dispone de un acelerómetro de menor rango para medir por separado el movimiento habitual. Esta elección

proporciona más margen de medida mientras se concretan las condiciones de vuelo [7, 8].

La LSM6DSV320X puede conectarse al SPI1 mediante el CS reservado para la IMU en el ejercicio anterior, sin modificar la asignación de pines. Para aprovechar ambos acelerómetros habría que leer también el canal de alto rango.

Cabe mencionar que, debido a la ausencia de suficientes datos como para poder afinar lo máximo posible el rango de medidas que nuestro sensor debería ser capaz de tomar, la elección queda justificada como punto de partida para la integración, pendiente de contrastar sus rangos y su configuración con los datos exactos de la misión a realizar.

1.4. Selección del receptor GNSS

El receptor GNSS proporcionará la posición geográfica del cohete para registrar su trayectoria y facilitar su recuperación. Antes del lanzamiento podrá obtener una posición inicial; durante el vuelo almacenaremos o transmitiremos las posiciones que el receptor indique como válidas. Su información complementará las medidas de la IMU y del barómetro, pero no será la única base para detectar el apogeo.

La misión de referencia tiene un apogeo previsto de unos 9 km sobre la rampa. Por tanto, el límite de altitud del receptor deberá superar la altitud absoluta que alcanzará el cohete, teniendo en cuenta también la elevación del lugar de lanzamiento. Debemos comprobar además sus límites de velocidad y aceleración. Como referencia histórica, Faraday publica para Origin una velocidad máxima de Mach 1,7 [1]. Su equivalencia en metros por segundo depende de la temperatura del aire en el punto donde se alcanzó, por lo que no podemos trasladar directamente ese dato al vehículo de este año. Para comparar receptores adoptaremos provisionalmente una velocidad máxima de 600 m/s.

Las estimaciones simplificadas realizadas con modelos de cohetes anteriores apuntan a aceleraciones del orden de 15 g durante la propulsión [5, 6]. Este valor sirve para examinar si un receptor podría mantener la posición durante esa fase, pero no constituye una aceleración máxima confirmada para la misión actual. Si el límite de un modelo es menor, estudiaremos si puede seguir proporcionando posiciones útiles después de la propulsión, especialmente durante el descenso. También tendremos en cuenta que, si pierde el seguimiento de los satélites, la recuperación de una posición válida puede requerir tiempo.

No exigiremos al GNSS la cadencia preliminar de 100 Hz planteada para las medidas inerciales y de presión: su frecuencia de actualización se escogerá según las necesidades de seguimiento y registro. El modelo seleccionado deberá poder comunicarse mediante el USART1 previsto en el ejercicio anterior, ser compatible con la alimentación de la placa y permitir instalar una antena con recepción adecuada.

1.4.1. Comparación de modelos

El Cuadro 3 compara tres receptores GNSS frente a una velocidad máxima provisional de 600 m/s y aceleraciones del orden de 15 g durante la propulsión. El segundo valor procede de estimaciones simplificadas sobre cohetes anteriores; ambos criterios deberán revisarse cuando se conozcan los datos del vehículo actual [5, 6].

El MAX-M10S es el más sencillo de incorporar a la arquitectura propuesta: dispone de UART y ocupa poco espacio. Sin embargo, sus límites operativos de 4 g y 500 m/s quedan por debajo de los valores adoptados para la comparación [9].

La configuración de Navsight-S consultada amplía el límite de aceleración hasta 10 g, pero man-

Criterio	MAX-M10S	Navsight-S	HDR1000 (HDR1110)
Aceleración operativa publicada	Hasta 4 g	Hasta 10 g en la configuración documentada	De -20 a $+20$ g
Velocidad límite publicada	500 m/s	515 m/s	3000 m/s
Altitud límite publicada	80 km	No se toma aquí como criterio comparativo: debe comprobarse para la configuración concreta	25 km
Comunicación e integración	UART; compatible en principio con el USART1 previsto	Sistema de navegación con puertos serie RS-232/422; requiere adaptar la interfaz eléctrica	Salida serie LVTTTL; requiere comprobar niveles eléctricos y configuración
Alimentación e integración física	Módulo compacto	Sistema de navegación completo	Alimentación de 5 V; consumo publicado de hasta 1,6 W incluida la antena
Resultado frente a 600 m/s y 15 g	Queda por debajo de ambos criterios	Queda por debajo de ambos criterios	Cubre ambos valores según las especificaciones publicadas

Cuadro 3: Comparación preliminar de receptores GNSS con distintos límites de dinámica [9, 10, 11].

tiene un límite de velocidad de 515 m/s. Por tanto, tampoco cubre los dos criterios provisionales. Además, Navsight es un sistema de navegación completo y sus puertos RS-232/422 no pueden conectarse directamente a los pines USART del STM32 sin adaptar los niveles eléctricos. Las prestaciones deberán asociarse a la configuración y revisión concretas del equipo [10].

El HDR1000, en su variante HDR1110, publica límites de ± 20 g, 3000 m/s y 25 km. Es el único de los tres que cubre los valores provisionales de aceleración, velocidad y altitud para la misión de 9 km. Su incorporación exigiría prever una alimentación de 5 V y comprobar la compatibilidad eléctrica de su salida serie con el STM32 [11].

Superar el límite de aceleración operativa de un receptor no significa que este se dañe ni que pierda necesariamente la posición en ese instante. Significa que su funcionamiento queda fuera de las condiciones especificadas por el fabricante: no podemos dar por fiables las posiciones de ese tramo sin verificarlas. Si pierde el seguimiento de los satélites, podrá volver a proporcionar una posición válida cuando recupere la señal y las condiciones vuelvan a estar dentro de sus límites, aunque no podemos asegurar que lo haga inmediatamente. El firmware deberá comprobar el estado de la solución GNSS y evitar utilizar posiciones antiguas como si fueran actuales.

1.4.2. Elección del receptor GNSS

Al tomar como referencia provisional una velocidad máxima de 600m/s, el MAX-M10S y el Navsight quedan fuera de sus límites de funcionamiento publicados, de 500 y 515 m/s, respectivamente. La aceleración también superaría sus límites durante la fase propulsada si se alcanzase la estimación preliminar de 15 g. Podríamos aceptar una pérdida temporal de medidas GNSS durante esa fase y volver a utilizar el receptor cuando la aceleración disminuyese, comprobando antes la validez de cada posición recibida. Sin embargo, el límite de velocidad podría seguir superándose después de terminar la propulsión. Por ello, ninguno de los dos permite justificar la adquisición GNSS a lo largo de toda la trayectoria prevista [9, 10].

Entre los receptores examinados no hemos encontrado uno de integración más sencilla que cubra tanto la velocidad provisional de 600 m/s como la aceleración estimada y que represente una alternativa intermedia al HDR1000. Seleccionamos, por tanto, el HDR1110 de esta familia como candidato para la integración: el fabricante publica una velocidad máxima de 3000 m/s, un intervalo de aceleración de $\pm 20g$ y una salida de datos a 10 Hz, aunque esta elección deberá revisarse al saber los datos exactos.

El STM32L476RGT6 dispone de USART1 en los pines PA9 (TX) y PA10 (RX), por lo que puede utilizar el periférico reservado para el GNSS en el ejercicio anterior. El HDR1110 requiere alimentación de 5 V y ofrece puertos serie LVTTTL. Para conectarlo habría que alimentar el receptor desde una línea de 5 V, compartir la masa y cruzar las señales TX y RX de ambos dispositivos. Aunque PA9 y PA10 figuran como pines tolerantes a 5 V, todavía no podemos confirmar una conexión directa: la información publicada del HDR1110 no indica las tensiones exactas que entrega y acepta su puerto serie. En particular, debemos comprobar que reconoce el nivel alto transmitido por el STM32 cuando este funciona a 3,3 V; si los niveles no son compatibles, se añadirá una adaptación de señales [12].

1.5. Resumen de la selección

Para el ejercicio se seleccionan dos unidades del barómetro LPS22DF, la IMU LSM6DSV320X y el receptor GNSS HDR1110. El Cuadro 4 resume las funciones de estos dispositivos y los periféricos previstos para conectarlos al STM32L476RGT6. Los dos barómetros comparten el mismo bus I²C1.

Función	Modelo seleccionado	Conexión prevista
Presión atmosférica	LPS22DF	I ² C1, mediante PB6 y PB7
Aceleración y velocidad angular	LSM6DSV320X	SPI1, con CS independiente en PA4
Posición geográfica	HDR1110	USART1, mediante PA9 y PA10; compatibilidad eléctrica de la conexión pendiente de confirmar

Cuadro 4: Sensores seleccionados e interfaces previstas con el STM32L476RGT6.

El LPS22DF cubre la presión estimada para la misión nominal de 9 km. La LSM6DSV320X permite medir por separado aceleraciones de menor y mayor amplitud, mientras se concretan las condiciones dinámicas del vehículo. El HDR1110 es el candidato examinado cuyos límites publicados incluyen las referencias provisionales de 600 m/s y 15 g.

2. Integración de los sensores

Partiremos del proyecto STM32CubeMX compilado en el ejercicio 1, que ya configura I²C1, SPI1 y USART1. Añadiremos las funciones necesarias para inicializar los dispositivos, comprobar que responden y obtener sus medidas a través de los buses correspondientes. El barómetro y la IMU se tratarán mediante lecturas de sus registros; para el receptor GNSS habrá que recibir e interpretar los mensajes enviados por su puerto serie.

2.1. Integración del LPS22DF

Antes de definir las conexiones, la figura 1 muestra los pines del LPS22DF y sus funciones. El esquema está representado en vista inferior. En nuestra conexión I²C serán especialmente importantes SCL, SDA, SA0 y CS, además de los pines de alimentación y masa [3].

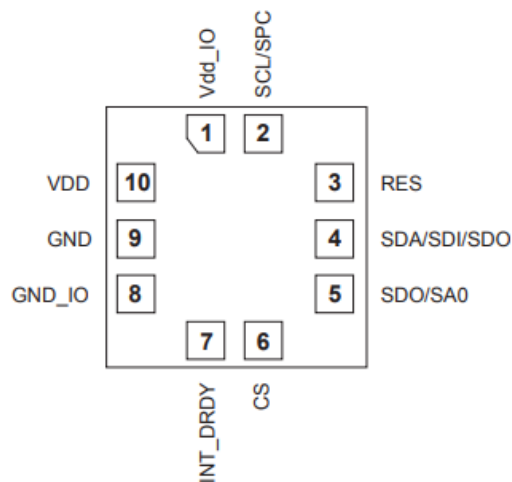


Table 1. Pin description

Pin number	Name	Function
1	Vdd_IO	Power supply for I/O pins
2	SCL SPC	I ² C / MIPI I3C SM serial clock (SCL) SPI serial port clock (SPC)
3	RES	Connect to GND
4	SDA SDI SDI/SDO	I ² C / MIPI I3C SM serial data (SDA) 4-wire SPI serial data input (SDI) 3-wire serial data input/output (SDI/SDO)
5	SDO SA0	4-wire SPI serial data output (SDO) I ² C least significant bit of the device address (SA0) MIPI I3C SM least significant bit of the static address (SA0)
6	CS	Enables SPI I ² C and MIPI I3C SM / SPI mode selection (1: SPI idle mode / I ² C and MIPI I3C SM communication enabled; 0: SPI communication mode / I ² C and MIPI I3C SM disabled)
7	INT_DRDY	Interrupt or data-ready
8	GND_IO	0 V supply
9	GND	0 V supply
10	VDD	Power supply

Figura 1: Pines y funciones del LPS22DF. Fuente: STMicroelectronics, figura 3 y tabla 1 de la ficha técnica [3].

El LPS22DF permite seleccionar su dirección I²C mediante el pin SA0. Si se conecta a masa, el dispositivo responde en 0x5C; si se conecta a VDD_IO, responde en 0x5D. Esta característica permite utilizar dos LPS22DF en el mismo bus I²C1, compartiendo PB6 y PB7 sin ocupar más pines del microcontrolador [3].

Aprovecharemos esta posibilidad para adquirir dos medidas de presión por separado. El firmware identificará, configurará y leerá cada barómetro, indicando si ha proporcionado una muestra nueva en el ciclo. Cuando ambos la proporcionen, calculará provisionalmente su media; cuando solo uno la proporcione, utilizará esa lectura. Esta operación no comprueba la concordancia entre sensores y no permite identificar automáticamente una medida errónea.

La configuración de STM32CubeMX del ejercicio anterior ya dispone de I²C1 en PB6 (SCL) y PB7 (SDA). Los dos barómetros compartirán estas líneas, por lo que no es necesario asignar nuevos pines ni modificar la configuración del periférico. Las líneas SCL y SDA necesitarán resistencias de *pull-up* compartidas hacia VDD_IO, además de alimentación y masa para cada dispositivo.

Para habilitar la comunicación I²C, el pin CS de ambos LPS22DF deberá mantenerse a nivel alto. Conectaremos SA0 del primero a masa y SA0 del segundo a VDD_IO, obteniendo las direcciones 0x5C y 0x5D, respectivamente. Estas conexiones pertenecen a los sensores y no requieren señales adicionales del STM32 [3].

La inicialización, identificación y lectura de los dos LPS22DF se muestran en el Anexo A. La presión combinada se utiliza para calcular la altura. La primera muestra válida establece la presión de referencia, correspondiente a una altura relativa de cero. Después, cada muestra nueva actualiza la altura filtrada y la lógica de detección de descenso. Si en un ciclo no se obtiene ninguna muestra nueva, estos valores no se actualizan. El programa se ha compilado correctamente en el proyecto STM32, pero todavía no se ha probado con los sensores físicos.

2.2. Integración del LSM6DSV320X

El LSM6DSV320X integra un acelerómetro triaxial de bajo rango, otro de alto rango y un giróscopo triaxial. Utilizaremos su interfaz SPI principal para comunicarlo con el STM32. La Figura 2 muestra la disposición de los pines en vista inferior, y el Cuadro 5 recoge sus funciones [8].

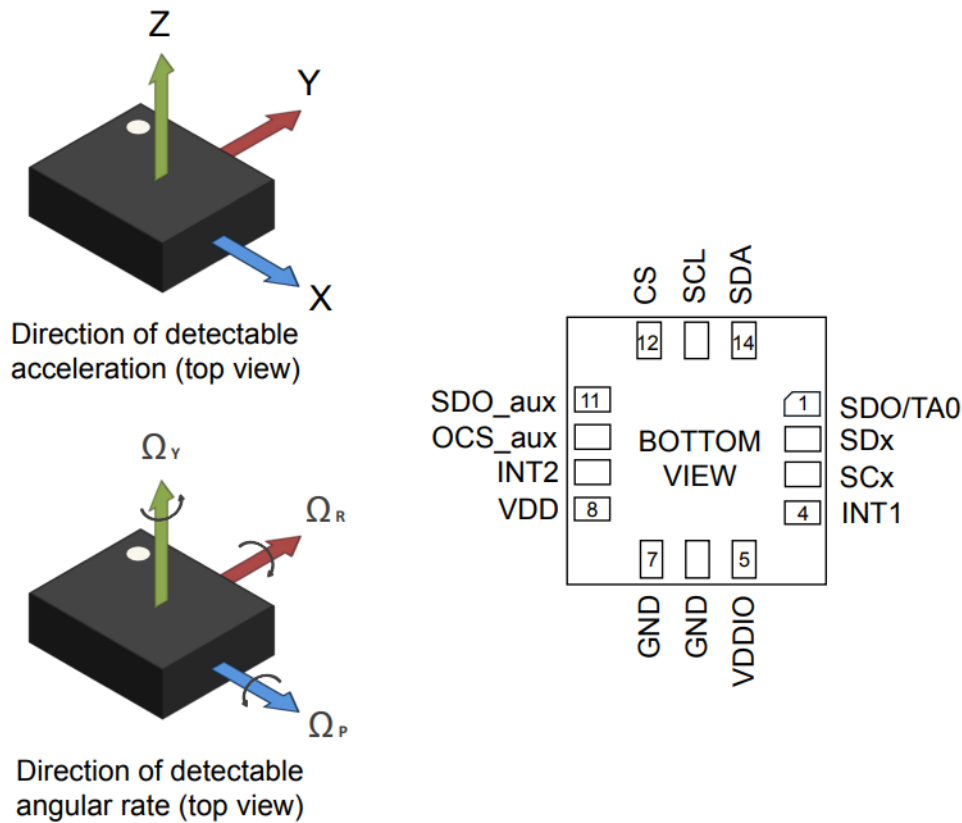


Figura 2: Pines del LSM6DSV320X, en vista inferior. Fuente: STMicroelectronics, figura 5 de la ficha técnica [8].

Para la comunicación SPI de cuatro hilos, conectaremos SCL/SPC al reloj de SPI1, SDA/SDI a MOSI y SDO/TA0 a MISO. El pin CS se controlará mediante la señal independiente CS_IMU del STM32: permanecerá alto cuando la IMU no esté seleccionada y se llevará a nivel bajo durante sus transacciones. También se conectarán VDD, VDDIO y los pines de masa conforme a las tensiones del circuito [8].

SPI1 ya está inicializado en el proyecto del ejercicio anterior y comparte sus líneas de reloj y datos con la memoria externa. Cada dispositivo dispone de su propia señal de selección, de modo que el programa podrá comunicarse con uno u otro en transacciones separadas. Por ahora emplearemos consultas desde el programa; los pines de interrupción de la IMU y su interfaz auxiliar no intervienen en esta primera integración.

El primer paso del firmware será leer el identificador del LSM6DSV320X para comprobar que la comunicación SPI funciona. Después se configurarán y leerán los canales de aceleración y velocidad angular necesarios para el ejercicio.

El acelerómetro *high-g* se configura con un rango de $\pm 320g$ y una frecuencia de salida de 480 Hz. El programa escribe el registro de configuración y comprueba su valor mediante una lectura posterior. Sin embargo, no adquiere ni procesa todavía las muestras de este canal. Dado que el bloque de adquisición se ejecuta cada 10 ms, una lectura por ciclo perdería muestras intermedias y podría omitir picos breves de aceleración. Para utilizar este canal en la detección de impactos habría que incorporar la FIFO del sensor o diseñar una adquisición que garantice la captura de esos picos. Esta ampliación queda fuera del alcance del ejercicio.

Pin	Nombre	Función en nuestra integración
1	SD0/TA0	Salida de datos hacia MISO en SPI de cuatro hilos.
2	SDx	Línea de la interfaz secundaria, que no utilizamos. En modo 1 se conecta a VDDIO o a GND.
3	SCx	Reloj de la interfaz secundaria, que no utilizamos. En modo 1 se conecta a VDDIO o a GND.
4	INT1	Salida de interrupción programable; no se utiliza en la primera implementación por consulta.
5	VDDIO	Alimentación de los pines de entrada y salida.
6,7	GND	Masa.
8	VDD	Alimentación del dispositivo.
9	INT2	Segunda salida de interrupción programable; no se utiliza inicialmente.
10	OCS_aux	Selección de la interfaz auxiliar. En modo 1 se conecta a VDDIO o se deja sin conexión eléctrica.
11	SD0_aux	Salida de la interfaz auxiliar. En modo 1 se conecta a VDDIO o se deja sin conexión eléctrica.
12	CS	Selección de la IMU: nivel bajo durante una transacción SPI y alto cuando no está seleccionada.
13	SCL/SPC	Entrada de reloj SPI; se conecta a SCK de SPI1.
14	SDA/SDI	Entrada de datos SPI; se conecta a MOSI de SPI1.

Cuadro 5: Funciones de los pines del LSM6DSV320X en el modo 1. Adaptado de la tabla 2 de STMicroelectronics [8].

2.3. Integración del receptor GNSS HDR1110

El HDR1110 es un receptor GNSS de la familia HDR1000. Recibe señales GPS L1C/A y BeiDou B1I y proporciona datos de navegación con una frecuencia de actualización de hasta 10 Hz. La Figura 3 resume sus interfaces externas y el Cuadro 6 explica la función de cada una. La figura es un esquema funcional: la documentación pública consultada no indica la numeración ni la disposición física de los contactos.

Interfaz	HDR1110
Alimentación de 5 V y masa	Receptor GNSS
Entrada RF mediante SMA(F)	Recepción de la antena
Puerto serie 1	Intercambio de datos
Puerto serie 2	Intercambio de datos
PPS	Referencia temporal

Figura 3: Interfaces funcionales del HDR1110. Esquema propio a partir de las especificaciones publicadas por el fabricante; no representa la disposición física de los contactos.

Interfaz	Función
Alimentación y masa	Suministran al receptor su tensión de entrada de 5 V y la referencia eléctrica común.
Entrada RF SMA(F)	Recibe la señal procedente de la antena GNSS.
Puerto serie 1	Permite el intercambio de datos con otro dispositivo. El fabricante ofrece una variante LVTTL y otras opciones de interfaz.
Puerto serie 2	Segunda interfaz de comunicación serie; su variante publicada es LVTTL o RS422.
PPS	Proporciona una señal periódica de referencia temporal.

Cuadro 6: Funciones de las interfaces del HDR1110.

En este proyecto se ha reservado USART1 del STM32 para recibir los datos del HDR1110. El firmware inicia una recepción por interrupción y guarda cada byte recibido en un búfer circular de 256 posiciones. Un contador registra cuántos bytes han llegado. Al llenarse el búfer, los datos nuevos sobrescriben los más antiguos.

La recepción se ha compilado, pero no se ha probado con el receptor físico. La configuración de USART1 a 115200 baudios es provisional: antes de conectar ambos equipos hay que confirmar la velocidad y los niveles eléctricos de la variante concreta del HDR1110. El programa aún no interpreta los mensajes ni extrae una posición GNSS. Para completar esa función habría que confirmar la configuración serie y el formato de salida del HDR1110, identificar y validar sus mensajes y extraer de ellos las coordenadas. También sería necesario ensayar la comunicación con el receptor físico.

2.4. Alcance de la implementación

El proyecto STM32 se ha compilado con la integración de los tres tipos de dispositivos seleccionados. El programa intenta identificar y configurar los dos LPS22DF, comprueba si cada uno dispone de una muestra nueva y combina las presiones recibidas. Con una presión combinada válida establece la referencia inicial, calcula la altura relativa y actualiza el filtrado y la detección de descenso. Si no llega una muestra nueva, estos cálculos no avanzan.

En la LSM6DSV320X, el programa identifica el dispositivo, configura y lee el acelerómetro de bajo rango y el giróscopo, y distingue las muestras nuevas de los valores conservados de ciclos anteriores. También configura y comprueba el canal de alto rango, pero todavía no adquiere sus muestras ni lo utiliza para detectar impactos.

Para el HDR1110 se ha habilitado la recepción por interrupción en USART1. Los bytes recibidos se almacenan en un búfer circular, pero el programa no separa ni valida mensajes GNSS y no obtiene coordenadas. La velocidad serie configurada es provisional hasta conocer los parámetros de la variante concreta del receptor.

La compilación confirma que el proyecto puede construirse, pero no verifica el funcionamiento con los dispositivos físicos ni las cadencias reales de adquisición. Las transacciones I²C y SPI empleadas son bloqueantes. Quedan pendientes las pruebas con los sensores, la comprobación eléctrica y del protocolo del GNSS, y una adquisición apropiada para las muestras del canal *high-g*.

Referencias

- [1] Faraday Rocketry UPV. (s.f.). *Faraday Rocketry UPV: Misiones*.
<https://faraday.webs.upv.es/>
- [2] NASA Glenn Research Center. (s.f.). *Earth atmosphere equation – metric*.
<https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/earth-atmosphere-equation-metric/>
- [3] STMicroelectronics. (2025). *LPS22DF: Low-power and high-precision MEMS nano pressure sensor* (DS13316, Rev. 3).
<https://www.st.com/resource/en/datasheet/lps22df.pdf>
- [4] TE Connectivity. (2017). *MS5611-01BA03: Barometric pressure sensor, with stainless steel cap* (Rev. B3). https://www.te.com/commerce/DocumentDelivery/DDEController?Action=showdoc&DocId=Data+Sheet%7FMS5611-01BA03%7FB3%7Fpdf%7FEnglish%7FENG_DS_MS5611-01BA03_B3.pdf%7FCAT-BLPS0036
- [5] RocketPy. *GENESIS – Faraday Rocketry UPV – 2023*.
https://docs.rocketpy.org/en/latest/examples/genesis_flight_sim.html
- [6] RocketPy. *Astra – Faraday Rocketry UPV – 2022*.
https://docs.rocketpy.org/en/develop/examples/astra_flight_sim.html
- [7] STMicroelectronics. *LSM6DSO32: iNEMO inertial module, always-on 3D accelerometer and 3D gyroscope* (DocID032891, Rev. 1).
<https://www.st.com/resource/en/datasheet/lsm6dso32.pdf>
- [8] STMicroelectronics. *LSM6DSV320X: 6-axis IMU with dual accelerometer up to 320 g* (DS14623). <https://www.st.com/resource/en/datasheet/lsm6dsv320x.pdf>
- [9] u-blox. *MAX-M10S: Data sheet* (UBX-20035208, Rev. R08). https://content.u-blox.com/sites/default/files/MAX-M10S_DataSheet_UBX-20035208.pdf
- [10] SBG Systems. *Navsight Solution: Hardware Manual* (NAVSIGHTHM, versión 1.7).
<https://support.sbg-systems.com/sc/hp/files/latest/108533453/108533760/1/1742981199475/NAVSIGHTHM%2B-%2Bv1.7.pdf>
- [11] GNSS Simulator. *HDR Series: High-Dynamic GNSS Receivers*, especificaciones publicadas para HDR1000/HDR1110.
<https://gnsssimulator.com/products/gnss-reception-monitoring/hdr-series/>
- [12] STMicroelectronics. (2024). *STM32L476xx: Ultra-low-power Arm Cortex-M4 32-bit MCU+FPU* (DS10198, Rev. 11).
<https://www.st.com/resource/en/datasheet/stm32l476rg.pdf>

A. Código de inicialización y lectura de los barómetros

Los siguientes fragmentos corresponden al programa que se comunica con dos LPS22DF mediante I²C1. El primero utiliza la dirección de 7 bits 0x5C y el segundo 0x5D, determinadas por la conexión de sus pines SA0. Las lecturas se combinan provisionalmente y se emplean para calcular una altura relativa a la presión registrada en la primera muestra válida [3].

A.1. Identificación de los dispositivos

Al arrancar, el programa lee el registro WHO_AM_I de cada dirección. Un dispositivo se considera detectado únicamente si la transacción I²C termina correctamente y devuelve 0xB4, el identificador indicado para el LPS22DF. La HAL recibe las direcciones I²C de 7 bits desplazadas una posición a la izquierda.

Listado 1: Lectura del identificador de los dos LPS22DF.

```
static volatile uint8_t baro_5c_detectado = 0U;
static volatile uint8_t baro_5d_detectado = 0U;

uint8_t id_5c = 0U;
uint8_t id_5d = 0U;

HAL_StatusTypeDef lectura_5c = HAL_I2C_Mem_Read(
    &hi2c1, 0x5CU << 1, 0x0FU, I2C_MEMADD_SIZE_8BIT,
    &id_5c, 1U, 100U
);

HAL_StatusTypeDef lectura_5d = HAL_I2C_Mem_Read(
    &hi2c1, 0x5DU << 1, 0x0FU, I2C_MEMADD_SIZE_8BIT,
    &id_5d, 1U, 100U
);

baro_5c_detectado =
    (lectura_5c == HAL_OK && id_5c == 0xB4U);
baro_5d_detectado =
    (lectura_5d == HAL_OK && id_5d == 0xB4U);
```

A.2. Configuración de la adquisición

El registro CTRL_REG1 recibe 0x3B: los bits ODR[3:0] seleccionan una frecuencia de salida de 100 Hz y los bits AVG[2:0] seleccionan un promediado de 32. Después de escribirlo, el programa lee el registro de nuevo y comprueba su contenido. También escribe 0x08 en CTRL_REG2 para activar BDU, que evita actualizar los registros de salida mientras se completa su lectura. Esta última escritura se comprueba mediante el estado devuelto por la HAL, pero el programa todavía no lee de vuelta CTRL_REG2 [3].

Listado 2: Configuración de ambos barómetros durante el arranque.

```
static volatile uint8_t baro_5c_configurado = 0U;
static volatile uint8_t baro_5d_configurado = 0U;

uint8_t configuracion = 0x3BU;      /* ODR 100 Hz, AVG 32 */
uint8_t configuracion_bdu = 0x08U;  /* Activa BDU */
uint8_t ctrl_5c = 0U;
uint8_t ctrl_5d = 0U;
```

```

if (baro_5c_detectado)
{
    baro_5c_configurado = (HAL_I2C_Mem_Write(
        &hi2c1, 0x5CU << 1, 0x10U, I2C_MEMADD_SIZE_8BIT,
        &configuracion, 1U, 100U
    ) == HAL_OK);
}

if (baro_5d_detectado)
{
    baro_5d_configurado = (HAL_I2C_Mem_Write(
        &hi2c1, 0x5DU << 1, 0x10U, I2C_MEMADD_SIZE_8BIT,
        &configuracion, 1U, 100U
    ) == HAL_OK);
}

if (baro_5c_configurado)
{
    HAL_StatusTypeDef resultado = HAL_I2C_Mem_Read(
        &hi2c1, 0x5CU << 1, 0x10U, I2C_MEMADD_SIZE_8BIT,
        &ctrl_5c, 1U, 100U
    );

    baro_5c_configurado =
        (resultado == HAL_OK && ctrl_5c == configuracion);
}

if (baro_5d_configurado)
{
    HAL_StatusTypeDef resultado = HAL_I2C_Mem_Read(
        &hi2c1, 0x5DU << 1, 0x10U, I2C_MEMADD_SIZE_8BIT,
        &ctrl_5d, 1U, 100U
    );

    baro_5d_configurado =
        (resultado == HAL_OK && ctrl_5d == configuracion);
}

if (baro_5c_configurado)
{
    baro_5c_configurado = (HAL_I2C_Mem_Write(
        &hi2c1, 0x5CU << 1, 0x11U, I2C_MEMADD_SIZE_8BIT,
        &configuracion_bdu, 1U, 100U
    ) == HAL_OK);
}

if (baro_5d_configurado)
{
    baro_5d_configurado = (HAL_I2C_Mem_Write(
        &hi2c1, 0x5DU << 1, 0x11U, I2C_MEMADD_SIZE_8BIT,
        &configuracion_bdu, 1U, 100U
    ) == HAL_OK);
}

```

A.3. Lectura de la presión

Dentro del bloque que se ejecuta cuando han transcurrido al menos 10 ms, el programa consulta el bit P_DA del registro STATUS de cada sensor. Si hay una muestra nueva, lee en una sola transacción PRESS_OUT_XL, PRESS_OUT_L y PRESS_OUT_H, en ese orden. La lectura termina en PRESS_OUT_H, como requiere BDU. Los tres bytes forman un entero con signo de 24 bits que se divide entre 4096 para obtener la presión en hPa [3].

El Listado 3 muestra el código del sensor 0x5C. El sensor 0x5D se lee de la misma manera, con su propia dirección y variables.

Listado 3: Consulta de datos nuevos y lectura de presión del LPS22DF con dirección 0x5C.

```
static volatile float baro_5c_presion_hpa = 0.0f;
static volatile uint8_t baro_5c_presion_valida = 0U;

/* Dentro del bloque de procesamiento de 10 ms: */
uint8_t nueva_presion_5c = 0U;

if (baro_5c_configurado)
{
    uint8_t estado_5c = 0U;

    HAL_StatusTypeDef resultado = HAL_I2C_Mem_Read(
        &hi2c1, 0x5CU << 1, 0x27U, I2C_MEMADD_SIZE_8BIT,
        &estado_5c, 1U, 100U
    );

    /* P_DA es el bit 0 de STATUS. */
    nueva_presion_5c =
        (resultado == HAL_OK && (estado_5c & 0x01U) != 0U);
}

/* Se reinicia la validez de la muestra en cada ciclo. */
baro_5c_presion_valida = 0U;

if (nueva_presion_5c)
{
    uint8_t bytes_presion[3] = {0U, 0U, 0U};

    HAL_StatusTypeDef lectura_presion = HAL_I2C_Mem_Read(
        &hi2c1, 0x5CU << 1, 0x28U, I2C_MEMADD_SIZE_8BIT,
        bytes_presion, 3U, 100U
    );

    if (lectura_presion == HAL_OK)
    {
        uint32_t bruto_24 =
            (uint32_t)bytes_presion[0]
            | ((uint32_t)bytes_presion[1] << 8)
            | ((uint32_t)bytes_presion[2] << 16);

        int32_t bruto_con_signo = (int32_t)bruto_24;
        if ((bruto_24 & 0x800000U) != 0U)
        {
            bruto_con_signo -= 0x1000000;
        }

        baro_5c_presion_hpa =
            (float)bruto_con_signo / 4096.0f;
    }
}
```

```

        baro_5c_presion_valida = 1U;
    }
}

```

El indicador de validez se reinicia en cada ciclo: conservar el último valor numérico de presión no significa que se haya obtenido una muestra nueva. Las llamadas I²C utilizadas son bloqueantes y admiten hasta 100 ms de espera por transacción. Por tanto, configurar una frecuencia de salida de 100 Hz y compilar el programa no demuestra que la adquisición efectiva mantenga esa cadencia.

A.4. Combinación provisional de las lecturas

El programa calcula la media cuando los dos barómetros han entregado una muestra nueva durante el mismo ciclo de procesamiento. Si solo uno ha entregado una muestra nueva, utiliza esa lectura e identifica su origen. Cuando no llega ninguna, conserva el valor numérico anterior, pero marca que no se ha obtenido una nueva presión combinada.

Listado 4: Combinación provisional de las presiones recibidas.

```

static volatile float presion_combinada_hpa = 0.0f;
static volatile uint8_t barometros_usados = 0U;

/* Tras intentar leer los dos sensores en el ciclo de 10 ms: */
barometros_usados = 0U;

if (baro_5c_presion_valida && baro_5d_presion_valida)
{
    presion_combinada_hpa =
        0.5f * (baro_5c_presion_hpa + baro_5d_presion_hpa);
    barometros_usados = 3U;
}
else if (baro_5c_presion_valida)
{
    presion_combinada_hpa = baro_5c_presion_hpa;
    barometros_usados = 1U;
}
else if (baro_5d_presion_valida)
{
    presion_combinada_hpa = baro_5d_presion_hpa;
    barometros_usados = 2U;
}

```

El valor de `barometros_usados` indica si se ha recibido una muestra nueva de ninguno de los sensores (0), solo del 0x5C (1), solo del 0x5D (2) o de ambos (3). Una lectura ausente en un ciclo no demuestra que el sensor haya fallado. La media provisional no comprueba la concordancia de las medidas ni garantiza una mayor exactitud.

A.5. Cálculo de la altura con la presión medida

El cálculo se realiza después de intentar leer los dos barómetros y combinar las muestras nuevas. La presión recibida en hPa se convierte a Pa antes de llamar a `AlturaDesdePresion`. La primera muestra válida fija la presión de referencia y, por tanto, una altura relativa inicial de cero. Si no llega ninguna muestra nueva, no avanzan el filtro, el historial ni la lógica de detección de descenso.

Listado 5: Uso de la presión combinada para actualizar la altura.

```

/* Variables persistentes dentro del bucle principal. */
static uint8_t referencia_pendiente = 1U;
static float presion_inicio_pa = 0.0f;
static uint8_t primera_medida = 1U;
static float altura_filtrada = 0.0f;

/* Despues de combinar las lecturas de ambos sensores: */
if (barometros_usados != 0U && presion_combinada_hpa > 0.0f)
{
    float presion_actual_pa = presion_combinada_hpa * 100.0f;

    if (referencia_pendiente)
    {
        presion_inicio_pa = presion_actual_pa;
        referencia_pendiente = 0U;
    }

    float altura_nueva =
        AlturaDesdePresion(presion_actual_pa, presion_inicio_pa);

    if (primera_medida)
    {
        altura_filtrada = altura_nueva;
        primera_medida = 0U;
    }
    else
    {
        altura_filtrada = FiltrarAltura(altura_nueva,
                                         altura_filtrada);
    }

    /* A continuacion, dentro de este mismo if, se actualizan
       la altura maxima, el historial y la deteccion de descenso. */
}

```

La función `AlturaDesdePresion` y los criterios de detección de descenso se mantienen respecto al ejercicio anterior. En esta versión, las confirmaciones de descenso cuentan muestras válidas: si se pierden lecturas, 50 confirmaciones ya no equivalen necesariamente a 0,5 s. El programa se ha compilado en el proyecto STM32, pero aún debe probarse con sensores físicos.

B. Código de inicialización y lectura de la IMU

Los siguientes fragmentos se incorporaron al proyecto STM32 para comunicarse mediante SPI1 con la IMU LSM6DSV320X. El programa identifica el dispositivo, configura el acelerómetro de bajo rango (*low-g*), el acelerómetro de alto rango (*high-g*) y el giróscopo, y lee las medidas del *low-g* y del giróscopo. El canal *high-g* queda configurado, pero sus medidas todavía no se adquieren. El proyecto se ha compilado correctamente; la comunicación y las lecturas no se han comprobado con una IMU física [8].

B.1. Identificación del dispositivo

Después de inicializar SPI1, el programa selecciona la IMU poniendo su señal CS a nivel bajo y lee el registro `WHO_AM_I` (0x0F). En la transacción SPI, el bit más significativo de la dirección se

pone a uno para indicar lectura; por ello se envía 0x8F. El primer byte recibido se descarta y el segundo contiene el identificador. La IMU solo se considera detectada si la transacción termina correctamente y el identificador leído es 0x73 [8].

Listado 6: Identificación de la IMU LSM6DSV320X mediante SPI1.

```
static volatile uint8_t imu_id_leido = 0U;
static volatile uint8_t imu_detectada = 0U;

/* Despues de inicializar SPI1 y el GPIO de CS_IMU: */
uint8_t imu_tx[2] = {0x8FU, 0x00U};
uint8_t imu_rx[2] = {0U, 0U};

HAL_GPIO_WritePin(CS_IMU_GPIO_Port, CS_IMU_Pin, GPIO_PIN_RESET);

HAL_StatusTypeDef resultado_imu = HAL_SPI_TransmitReceive(
    &hspi1, imu_tx, imu_rx, 2U, 100U
);

HAL_GPIO_WritePin(CS_IMU_GPIO_Port, CS_IMU_Pin, GPIO_PIN_SET);

if (resultado_imu == HAL_OK)
{
    imu_id_leido = imu_rx[1];
    imu_detectada = (imu_id_leido == 0x73U);
}
```

B.2. Configuración de la interfaz

Una vez identificada la IMU, el programa escribe 0x44 en CTRL3 (0x12). Con ello activa BDU, que evita actualizar una medida mientras se leen sus bytes, e IF_INC, que permite leer registros consecutivos en una misma transacción. Después lee CTRL3 de vuelta y comprueba que ambos bits están activados. Solo si se supera esta comprobación continúa la configuración de los canales de medida [8].

Listado 7: Activación y comprobación de BDU e IF_INC.

```
static volatile uint8_t imu_interfaz_configurada = 0U;

/* Despues de identificar la IMU: */
if (imu_detectada)
{
    uint8_t imu_config_tx[2] = {0x12U, 0x44U};

    HAL_GPIO_WritePin(CS_IMU_GPIO_Port, CS_IMU_Pin, GPIO_PIN_RESET);
    HAL_StatusTypeDef escritura_imu = HAL_SPI_Transmit(
        &hspi1, imu_config_tx, 2U, 100U
    );
    HAL_GPIO_WritePin(CS_IMU_GPIO_Port, CS_IMU_Pin, GPIO_PIN_SET);

    if (escritura_imu == HAL_OK)
    {
        uint8_t imu_verif_tx[2] = {0x92U, 0x00U};
        uint8_t imu_verif_rx[2] = {0U, 0U};

        HAL_GPIO_WritePin(CS_IMU_GPIO_Port, CS_IMU_Pin,
            GPIO_PIN_RESET);
        HAL_StatusTypeDef lectura_imu = HAL_SPI_TransmitReceive(
            &hspi1, imu_verif_tx, imu_verif_rx, 2U, 100U
        );
    }
}
```

```

);
HAL_GPIO_WritePin(CS_IMU_GPIO_Port, CS_IMU_Pin,
                  GPIO_PIN_SET);

imu_interfaz_configurada =
    (lectura_imu == HAL_OK &&
     (imu_verif_rx[1] & 0x44U) == 0x44U);
}
}

```

B.3. Configuración del acelerómetro de bajo rango

Si la interfaz se ha configurado correctamente, el programa selecciona un rango de $\pm 16 g$ mediante CTRL8 y una frecuencia de salida de 60 Hz en modo de alto rendimiento mediante CTRL1. Después lee ambos registros de vuelta. El indicador `imu_lowg_configurado` solo se activa si las dos lecturas terminan correctamente y contienen los valores esperados [8].

Cuadro 7: Registros utilizados para configurar el acelerómetro *low-g*.

Registro	Dirección	Valor escrito	Función
CTRL8	0x17	0x03	Rango de $\pm 16 g$
CTRL1	0x10	0x05	Alto rendimiento, 60 Hz

B.4. Configuración de los otros canales de la IMU

El acelerómetro *high-g* se configura mediante CTRL1_XL_HG para un rango de $\pm 320 g$ y una frecuencia de salida de 480 Hz. El programa lee el registro después de escribirlo y comprueba que conserva el valor esperado. Este canal queda configurado, pero sus medidas no se adquieren todavía.

El giróscopo se configura con un rango de ± 2000 °/s mediante CTRL6 y una frecuencia de salida de 60 Hz mediante CTRL2. El programa lee ambos registros de vuelta antes de considerar completada su configuración [8].

Cuadro 8: Configuración del canal *high-g* y del giróscopo.

Canal	Registro	Valor escrito	Configuración
High-g	CTRL1_XL_HG	0x9C	$\pm 320 g$, 480 Hz
Giróscopo	CTRL6	0x04	± 2000 °/s
Giróscopo	CTRL2	0x05	60 Hz

B.5. Lectura de la aceleración y la velocidad angular

En cada paso del bloque de adquisición, el programa reinicia los indicadores de muestra válida y consulta STATUS_REG. El bit XLDA indica si hay una muestra nueva del acelerómetro *low-g*; el bit GDA cumple la misma función para el giróscopo. Si hay datos nuevos, se leen los seis bytes de X, Y y Z en una transacción SPI para cada canal [8].

Cuadro 9: Registros y conversión de las medidas de la IMU.

Canal	Bit de estado	Registros de salida	Conversión
Low-g	XLDA, bit 0	0x28-0x2D	0,000488 g/LSB
Giróscopo	GDA, bit 1	0x22-0x27	0,070 °/s/LSB

En ambos casos, cada eje se reconstruye a partir de un byte bajo y otro alto como un entero de 16 bits con signo. Los valores convertidos conservan la última muestra recibida, pero el indicador de validez solo se activa cuando se ha detectado una muestra nueva y la transacción SPI ha terminado correctamente. Por tanto, conservar un valor numérico no implica haber adquirido una medida nueva en ese paso. El Listado 8 muestra la consulta de datos nuevos y el comienzo de la lectura del acelerómetro *low-g*. Se presenta el eje X como ejemplo; en el programa, los ejes Y y Z se reconstruyen a partir de los cuatro bytes restantes. La lectura del giróscopo sigue el mismo procedimiento con sus propios registros, indicador de validez y factor de conversión.

Listado 8: Consulta de una muestra nueva y lectura del canal *low-g*; se muestra la conversión del eje X.

```
imu_lowg_muestra_valida = 0U;

if (imu_lowg_configurado)
{
    uint8_t estado_tx[2] = {0x9EU, 0x00U};
    uint8_t estado_rx[2] = {0U, 0U};

    HAL_GPIO_WritePin(CS_IMU_GPIO_Port, CS_IMU_Pin, GPIO_PIN_RESET);
    HAL_StatusTypeDef lectura_estado = HAL_SPI_TransmitReceive(
        &hspi1, estado_tx, estado_rx, 2U, 100U
    );
    HAL_GPIO_WritePin(CS_IMU_GPIO_Port, CS_IMU_Pin, GPIO_PIN_SET);

    if (lectura_estado == HAL_OK &&
        (estado_rx[1] & 0x01U) != 0U)
    {
        uint8_t datos_tx[7] = {0xA8U, 0U, 0U, 0U, 0U, 0U, 0U};
        uint8_t datos_rx[7] = {0U, 0U, 0U, 0U, 0U, 0U, 0U};

        HAL_GPIO_WritePin(CS_IMU_GPIO_Port, CS_IMU_Pin,
            GPIO_PIN_RESET);
        HAL_StatusTypeDef lectura_datos = HAL_SPI_TransmitReceive(
            &hspi1, datos_tx, datos_rx, 7U, 100U
        );
        HAL_GPIO_WritePin(CS_IMU_GPIO_Port, CS_IMU_Pin,
            GPIO_PIN_SET);

        if (lectura_datos == HAL_OK)
        {
            int16_t ax_bruta = (int16_t)(
                (uint16_t)datos_rx[1] |
                ((uint16_t)datos_rx[2] << 8)
            );

            imu_ax_g = (float)ax_bruta * 0.000488f;
            /* El programa tambien calcula Y y Z aqui. */
            imu_lowg_muestra_valida = 1U;
        }
    }
}
```

```
}
```

La configuración del canal *high-g* se comprueba mediante una lectura de su registro, pero el programa todavía no lee sus muestras. Por su frecuencia de salida de 480 Hz, consultar solo el valor más reciente cada 10 ms podría omitir picos breves; su adquisición requeriría un tratamiento adicional, como el uso de la FIFO.

Las transacciones SPI de este código son bloqueantes y tienen un tiempo máximo de espera de 100 ms cada una. Por ello, configurar frecuencias de salida de 60 Hz o 480 Hz no demuestra que el programa adquiera y procese datos a esas frecuencias. El proyecto se ha compilado, pero aún no se han verificado la comunicación SPI, las configuraciones ni las medidas con la IMU física.

C. Recepción de datos del receptor GNSS HDR1110

El siguiente código prepara la recepción de datos por USART1 y almacena los bytes recibidos. No interpreta todavía los mensajes del receptor ni calcula una posición GNSS.

Listado 9: Variables utilizadas para la recepción GNSS.

```
static uint8_t gnss_byte_rx = 0U;
static volatile uint8_t gnss_buffer[256] = {0U};
static volatile uint16_t gnss_indice = 0U;
static volatile uint32_t gnss_bytes_recibidos = 0U;
static volatile uint8_t gnss_rx_activa = 0U;
```

Tras inicializar USART1, el programa solicita la recepción del primer byte:

Listado 10: Inicio de la recepción por interrupción.

```
gnss_rx_activa =
    (HAL_UART_Receive_IT(&huart1, &gnss_byte_rx, 1U) == HAL_OK);
```

Cada vez que llega un byte, la función de retorno de la HAL lo guarda en el búfer circular y solicita el siguiente:

Listado 11: Almacenamiento de cada byte recibido por USART1.

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        gnss_buffer[gnss_indice] = gnss_byte_rx;
        gnss_indice = (uint16_t)((gnss_indice + 1U) & 0xFFU);
        gnss_bytes_recibidos++;

        gnss_rx_activa =
            (HAL_UART_Receive_IT(&huart1, &gnss_byte_rx, 1U) == HAL_OK);
    }
}
```

El búfer tiene 256 posiciones: después de la última, el índice vuelve a cero y las nuevas entradas sobrescriben las más antiguas. El contador registra bytes recibidos, no mensajes GNSS válidos. El código se ha compilado, pero todavía no se ha ensayado con el receptor físico.